Experiment 5

Naïve Bayes Classification for Text and Categorical Data using Prior and Posterior Probabilities

Aim: To implement the Naïve Bayes classifier based on Bayes' theorem for probabilistic classification and evaluate its predictive performance.

Theory:

Naive Bayes is a supervised probabilistic learning algorithm widely used for text classification tasks such as spam and ham message detection. The model is based on Bayes' theorem, which estimates the posterior probability of a class given observed features. A key assumption of Naive Bayes is that features are conditionally independent given the class label. While this assumption does not fully hold for natural language, the classifier remains effective due to the high dimensionality and sparse distribution of textual features.

1. Bayes' Theorem

Bayes' theorem is a fundamental probability rule that describes how to update the probability of a hypothesis based on new evidence. It forms the mathematical foundation of the Naive Bayes classifier. The theorem is expressed as:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where:

- **P(C | X)** represents the posterior probability of a message belonging to class C (spam or ham) given the feature set X.
- **P(X | C)** is the likelihood of observing the features under class C.
- **P(C)** denotes the prior probability of the class.
- **P(X)** is the marginal likelihood of the features.

Prior Probability

The prior probability P(C) represents the probability of a class occurring before observing any features. It is calculated from the training data as:

$$P(C) = \frac{\text{Number of samples belonging to class C}}{\text{Total number of samples}}$$

For example, in a spam detection dataset where 40% are spam and 60% are ham:

- P(Spam) = 0.4
- P(Ham) = 0.6

Likelihood

The likelihood P(X | C) represents the probability of observing a feature vector X when the class is C. In text classification, this corresponds to the probability of words appearing in messages belonging to a particular class.

Posterior Probability

The posterior probability P(C | X) is the updated probability of a class after considering the observed features. The classifier computes this value for each class and selects the class with the highest posterior probability:

$$C^* = \max P(C|X)$$

where C* is the predicted class. This approach is also referred to as **Maximum A Posteriori** (MAP) estimation, which chooses the class that maximizes the posterior probability:

$$\mathrm{MAP(C)} = \max P(X|C)P(C)$$

The classifier therefore only needs to compute the product of the likelihood and prior probability for each class and select the class with the highest value.

2. Naive Independence Assumption

The term **"Naive"** refers to the simplifying assumption that all features are conditionally independent given the class label. This means that the presence or absence of one feature does not influence another feature when the class is known.

If the feature vector is represented as:

$$X \;=\; (x_1, x_2, x_3, \dots, x_\mathrm{n})$$

then the likelihood can be simplified as:

$$P(X \mid C) \;=\; P(x_1 \mid C) \times P(x_2 \mid C) \times \dots \times P(x_\mathrm{n} \mid C)$$

This assumption significantly reduces computational complexity and allows the algorithm to work efficiently with large datasets. Instead of estimating the joint probability of all features together, which would require an exponentially large number of parameters, the model only needs to estimate the individual conditional probabilities of each feature. This makes Naive Bayes especially effective for high-dimensional data such as text.

3. Naive Bayes Classifier

The Naive Bayes classifier applies Bayes' theorem together with the naive independence assumption to classify new instances. Given an input feature vector X = (x_1, x_2, …, x_n), the classifier computes the posterior probability for each class and assigns the label of the class with the highest posterior probability.

The classification rule is:

$$C* \;=\; argmax\;\; P(C) \times \prod P(x_\mathrm{i} \mid C) \quad for\; i \;=\; 1\; to\; n$$

To avoid numerical underflow when dealing with very small probability values, logarithms are applied:

$$C* \;=\; argmax\;\; [\, log\; P(C) \;+\; \Sigma\; log\; P(x_\mathrm{i} \mid C)\,]$$

While the underlying principle remains consistent, different variants of the classifier are used depending on the nature of the data distribution and feature representation.

Multinomial Naive Bayes

Multinomial Naive Bayes is designed for discrete data, especially word counts or term frequencies. It is widely used in text classification tasks such as spam detection and document classification. Text is typically converted into numerical features using methods like bag-of-words or TF-IDF.

Gaussian Naive Bayes

Gaussian Naive Bayes is used when features are continuous numerical values assumed to follow a normal distribution. The likelihood is calculated using the Gaussian probability density function, using the mean and variance of the feature values for each class to measure how closely a value fits the distribution of that class.

Bernoulli Naive Bayes

Bernoulli Naive Bayes works with binary features, where each feature indicates whether a word is present (1) or absent (0) in a document. It is useful when the presence of certain keywords is more important than their frequency.

Categorical Naive Bayes

Categorical Naive Bayes is used when features are categorical variables with multiple possible values, such as colour, weather, or device type. Probabilities are calculated based on the frequency of each category within a class.

4. Feature Transformation and Modeling

Before applying the Naive Bayes classifier, raw data must be transformed into a suitable numerical representation depending on the nature of the features.

Categorical Features

Categorical features take discrete values from a fixed set (e.g., colour: red, blue, green). The probability of each category given a class is estimated from training data:

$$P(x_i = v \mid C = c) = Count(x_i = v, C = c) \ / \ Count(C = c)$$

Laplace smoothing is applied to avoid zero probabilities for unseen category values:

$$P(x_i = v \mid C = c) = (Count + 1) \ / \ (Total + Number\ of\ unique\ values)$$

Continuous Features (Gaussian NB)

When features are continuous, Gaussian Naive Bayes assumes values follow a normal distribution within each class. The mean (μ) and variance (σ^2) are estimated from training data for each feature and class. The probability density function computes the likelihood:

$$P(x_i \mid C) = (1 / \sqrt{(2\pi\sigma^2)}) \times exp(-(x_i - \mu)^2 / (2\sigma^2))$$

Frequency-Based Features

Textual data is transformed into numerical vectors using Term Frequency–Inverse Document Frequency (TF-IDF) weighting. TF-IDF enhances the importance of words that are frequent within a document but infrequent across the entire corpus, improving the discriminative ability of the feature space. For classification, the Multinomial Naive Bayes algorithm is well suited for word-based features and TF-IDF representations.

In text classification tasks, documents are represented using the bag-of-words model, where word occurrences form the basis of feature extraction and the order of words is ignored. For example, a message can be represented as shown in the table below:

Word	Frequency
free	2
win	1
offer	1

To improve feature quality and reduce redundancy, Natural Language Processing (NLP) techniques are applied. These include:

- **Text Normalization:** Standardizing text format.
- **Stop Words Removal:** Filtering out common words (e.g., "is", "the") that carry little semantic value.
- **Stemming and Lemmatization:** Reducing words to their base or root forms.
- **Part-of-Speech (POS) Tagging:** Identifying grammatical roles to preserve semantic correctness while reducing dimensionality.

5. Naïve Bayes Classification Workflow

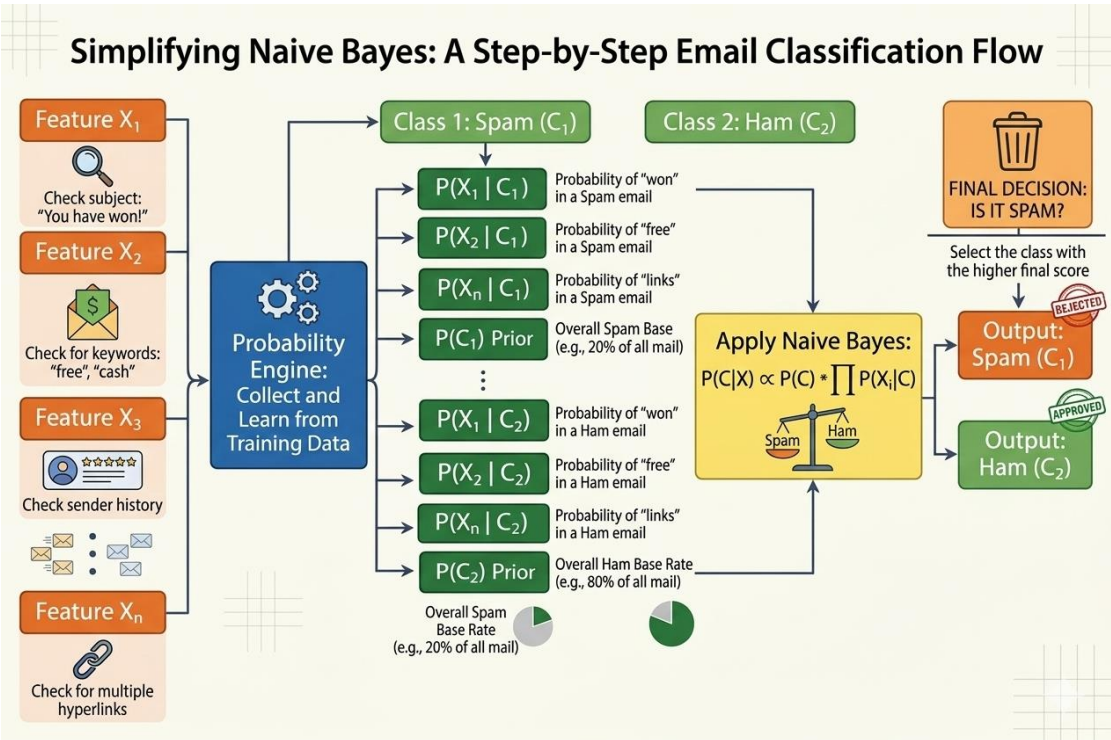


Figure 1: Workflow of Naïve Bayes classification for spam–ham email prediction

The figure 1 illustrates the workflow of the Naïve Bayes classifier used for email classification. In this process, an email is first analyzed by extracting several features such as the presence of certain keywords in the subject, specific terms in the message body, or patterns in the sender information. These extracted features represent the input variables that help characterize the email. Using training

data, the classifier estimates the probabilities of these features occurring in each class, namely spam and ham. It also computes the prior probabilities of each class, which represent how frequently spam and ham emails appear in the dataset.

Once these probabilities are learned, the Naïve Bayes algorithm applies Bayes' theorem to compute the posterior probability of the email belonging to each class given the observed features. This involves combining the prior probability of the class with the conditional probabilities of the features. The classifier then compares the resulting probability scores for the spam and ham classes and assigns the email to the class with the higher probability. This probabilistic approach enables efficient and accurate classification of emails into spam or legitimate messages.

6. Algorithm

1. **Step 1:** Apply Bayes' Theorem:
   - $P(Class \mid Features) = P(Features \mid Class) \times P(Class) / P(Features)$
2. **Step 2:** Apply Naive Independence Assumption:
   - $P(x_1, x_2, \ldots, x_n \mid Class) = P(x_1 \mid Class) \times P(x_2 \mid Class) \times \ldots \times P(x_n \mid Class)$
3. **Step 3:** During Training, calculate from data:
   - **Prior Probability:**
     - $P(Class = c) = Count\ of\ samples\ in\ class\ c\ /\ Total\ samples$
   - **Conditional Probability (for each feature):**
     - **For categorical features:** $P(x_i = v \mid Class = c) = Count(class\ c\ where\ feature\ j = v)\ /\ Count(class\ c\ samples)$
     - **For continuous features (Gaussian NB):**
     - Calculate mean (μ) and variance (σ^2) of feature j for each class
     - $P(x_i \mid Class = c) = (1 / \sqrt{(2\pi\sigma^2)}) \times exp(-(x_i - \mu)^2 / (2\sigma^2))$
4. **Step 4:** Add Laplace Smoothing (to handle zero probabilities):
   - $P(x_i = v \mid Class = c) = (Count + 1) / (Total + Number\ of\ unique\ values)$
5. **Step 5:** For Prediction:
   - For each class c, calculate: $Score(c) = P(Class = c) \times \prod P(x_i \mid Class = c)$
   - Use log to avoid underflow: $Log_Score(c) = log(P(Class = c)) + \Sigma\ log(P(x_i \mid Class = c))$
   - **Predict class with highest score**

7. Applications: Text Classification and NLP

Naive Bayes is particularly well suited for text classification problems. The high-dimensional and sparse nature of text data aligns naturally with the independence assumption, making the classifier both efficient and effective for NLP tasks.

Spam Detection

Naive Bayes is one of the earliest and most effective methods for email spam detection. Given a set of words in an email, the classifier computes the probability that the message belongs to the spam or ham class. Words such as "free", "offer", and "win" tend to have high probability under the spam class, while neutral vocabulary is more characteristic of ham messages. The model learns these probabilities from labelled training data and applies them to new, unseen messages.

Sentiment Analysis

In sentiment analysis, the goal is to determine whether a piece of text (such as a product review or a tweet) expresses a positive, negative, or neutral opinion. Naive Bayes can be trained on labelled sentiment data to estimate the likelihood of positive or negative words under each class. Due to its fast training speed, it is widely used as a baseline model for sentiment classification tasks.

Document Classification

Naive Bayes is also applied to categorize documents into predefined topics or genres, such as news articles into sports, politics, and technology. The bag-of-words or TF-IDF representation of documents is used as the feature set. The classifier assigns a document to the category with the highest posterior probability, and performs well when documents have clearly distinguishable vocabulary across categories.

In all these text-based applications, NLP preprocessing techniques such as stop word removal, stemming, and TF-IDF weighting further enhance the performance of the Naive Bayes classifier by reducing noise and emphasizing discriminative features.

8. Merits of Naive Bayes

- **Simple and Computationally Efficient:** The algorithm requires only a single pass through the training data to estimate probabilities, making training and prediction extremely fast.
- **Works Well with High-Dimensional Data:** Suitable when the number of features is very large, such as in text classification with thousands of words. The independence assumption prevents the curse of dimensionality.
- **Performs Well with Small Datasets:** Unlike many other machine learning algorithms, Naive Bayes can yield good results even with a small amount of training data, as it only requires individual feature probabilities.

9. Demerits of Naive Bayes

- **Strong Independence Assumption:** Naive Bayes assumes all features are conditionally independent given the class. In practice this rarely holds — words like "bank" and "loan" in a document are related, yet the model treats them as independent.
- **Performance Decreases When Features Are Highly Correlated:** When features are strongly correlated, the independence assumption is violated, leading to biased probability estimates and reduced classification accuracy.
- **Limited Ability to Capture Complex Relationships:** Naive Bayes is a simple linear classifier and cannot model non-linear patterns or complex feature interactions. More sophisticated algorithms such as decision trees, random forests, or neural networks may outperform it on complex datasets.

Overall, the Naive Bayes classifier, combined with NLP-based feature normalization and TF-IDF representation, provides an efficient and robust solution for spam and ham message classification.

Procedure:

The objective of this experiment is to classify the text messages into two categories—Spam and Ham. The input to the model consists of multiple independent random text messages, while the output is a binary dependent variable indicating the category of that message.

Step 1: Import required Libraries: pandas, numpy, re, matplotlib, nltk, and the necessary modules from scikit-learn.

Step 2: Load Dataset: Load the Spam_Detection.csv file into a DataFrame using pd.read_csv(). The dataset contains 1742 text messages, where each row represents an individual message and its corresponding category (Spam or Ham).

Step 3: Exploratory Data Analysis: Conduct initial data exploration using head(), tail(), info(), shape, and value_counts() to understand the dataset structure and class distribution.

Step 4: Check Missing Values: Inspect the dataset for any missing entries using isnull().sum() to ensure data quality before processing.

Step 5: Visualize Class Distribution: Create visual representations, such as pie charts, to illustrate the balance between Spam and Ham messages in the dataset.

Step 6: Initialize Preprocessing Tools: Set up essential Natural Language Processing (NLP) tools, including the PorterStemmer and the standard English stop-word list.

Step 7: Text Preprocessing: Clean the text data by implementing functions to convert text to lowercase, remove URLs, strip special characters and numbers, and eliminate redundant white spaces.

Step 8: Target Encoding: Transform the categorical labels into numerical format, where Ham is encoded as 0 and Spam is encoded as 1.

Step 9: Data Partitioning: Split the preprocessed dataset into training (80%) and testing (20%) sets to facilitate model building and evaluation.

Step 10: TF-IDF Vectorization: Convert the textual messages into numerical features using the Term Frequency-Inverse Document Frequency (TF-IDF) vectorization technique.

Step 11: Model Training: Initialize and train a Multinomial Naive Bayes model using the vectorized training data.

Step 12: Performance Evaluation: Assess the model's effectiveness using metrics such as Accuracy Score, Precision, Recall, and F1-score, supplemented by a Confusion Matrix and Precision-Recall curve visualizations.

Step 13: Prediction Analysis: Evaluate the model's real-world applicability by analyzing predictions on random test samples, displaying the actual label, predicted label, and the calculated spam probability for each sample.

Naive Bayes - Practical Implementation

Step 1: Importing Libraries

Importing necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import HTML, display
print("Libraries Imported")
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

Read the Spam SMS dataset

```
data = pd.read_csv('spam_detection.csv', encoding='latin1', header=0, usecols=[0,1],
    engine="python")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Step 3: Data Analysis

Display first 5 rows

```
data.head()
```

Output:

	Category	Message
0	spam	Get your garden ready for summer with a free selection of summer bulbs and see..
1	spam	This is your chance to be on a reality fantasy show. Call now at 08707509020. ..
2	ham	Love is not a decision; it is a feeling. If we could choose whom to love, life..
3	ham	Good friends care for each other, close friends understand each other, and tru..
4	spam	Get your garden ready for summer with a free selection of summer bulbs and see..

Display Last 5 rows

```
data.tail()
```

Output:

	Category	Message
1737	spam	Do you want 750 anytime any network mins 150 text and a NEW VIDEO phone for only..
1738	ham	Thanx...
1739	spam	88800 and 89034 are premium phone services call 08718711108
1740	spam	Today's Offer! Claim ur £150 worth of discount vouchers! Text YES to 85023 now! ..
1741	ham	am up to my eyes in philosophy

Displays summary of dataset, including column names, data types, and non-null counts.

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1742 entries, 0 to 1741
Data columns (total 2 columns):
#   Column     Non-Null Count  Dtype
---  ---------  --------------- -----
0   Category   1742 non-null   object
1   Message    1742 non-null   object
dtypes: object(2)
memory usage: 27.3+ KB
```

Checks the dataset for missing(NULL) values in each column

```
data.isnull().sum()
```

Output:

```
Category    0
Message     0
dtype: int64
```

Shows the size of the dataset

```
data.shape
```

Output:

```
(1742, 2)
```

Counts the frequency of each unique category

```
data['Category'].value_counts()
```

Output:

Category	Count
Ham	1003
Spam	739

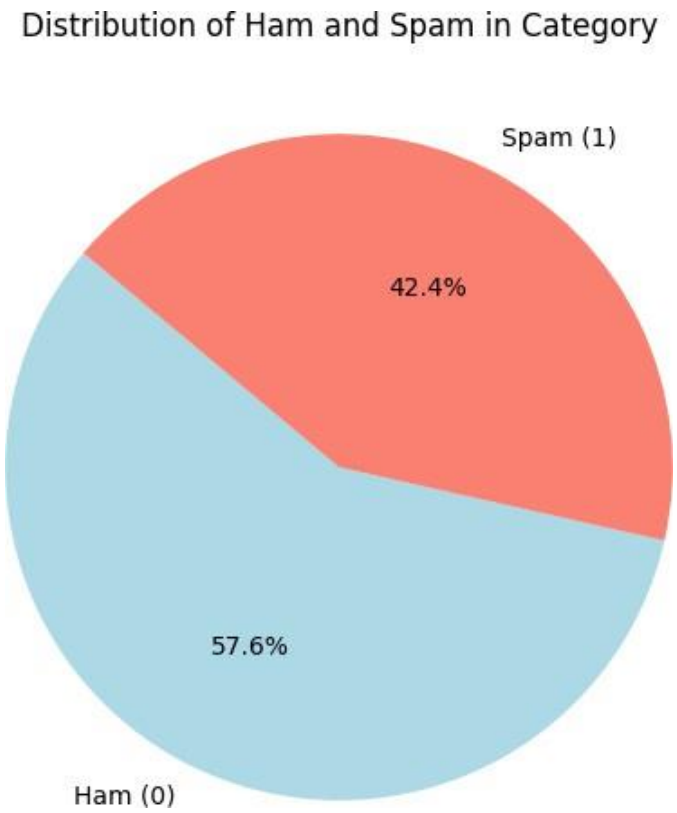
Output:
```
dtype: int64
```

Step 4: Data Preprocessing

 # Visualizes the distribution of spam and ham messages in the dataset

```
category_counts = data['Category'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(category_counts, labels=['Ham (0)', 'Spam (1)'],
    autopct='%1.1f%%', startangle=140, colors=['lightblue', 'salmon'])
plt.title('Distribution of Ham and Spam in Category');
plt.show()
```

Output:
```
Distribution of Ham and Spam in Category
```



 # Sets up stemming and stop-word removal tools for text preprocessing.

```
stemmer = PorterStemmer()
stop_words = set(ENGLISH_STOP_WORDS)
print("Initialized Porter Stemmer and English stopword list.")
```

Output:
```
Initialized Porter Stemmer and English stopword list.
```

 # Performs basic text cleaning and normalisation

```
def preprocess_text(s):
    if pd.isnull(s): return ""
    s = s.lower(); s = re.sub(r'http\S+|www\.\S+', ' ', s); s = re.sub(r'[^a-z0-9\s]', '
    ', s)
    s = re.sub(r'\s+', ' ', s).strip(); return s
print("Text Cleaned")
```

Output:

```
Text Cleaned
```

Step 5: Model Training

 # Encodes spam labels and splits the data into stratified training and testing sets.

```
data['label'] = (data['Category'].str.lower().str.strip() == 'spam').astype(int)
X_train, X_test, y_train, y_test = train_test_split( data['Message'], data['label'],
    test_size=0.25, random_state=42, stratify=data['label'] )
print("Labels encoded and data split into training and testing sets.")
```

Output:

```
Labels encoded and data split into training and testing sets.
```

 # Converts text messages into TF-IDF feature vectors.

```
vectorizer = TfidfVectorizer(ngram_range=(1,3), min_df=2, max_features=50000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
print("Text data vectorized using TF-IDF.")
```

Output:

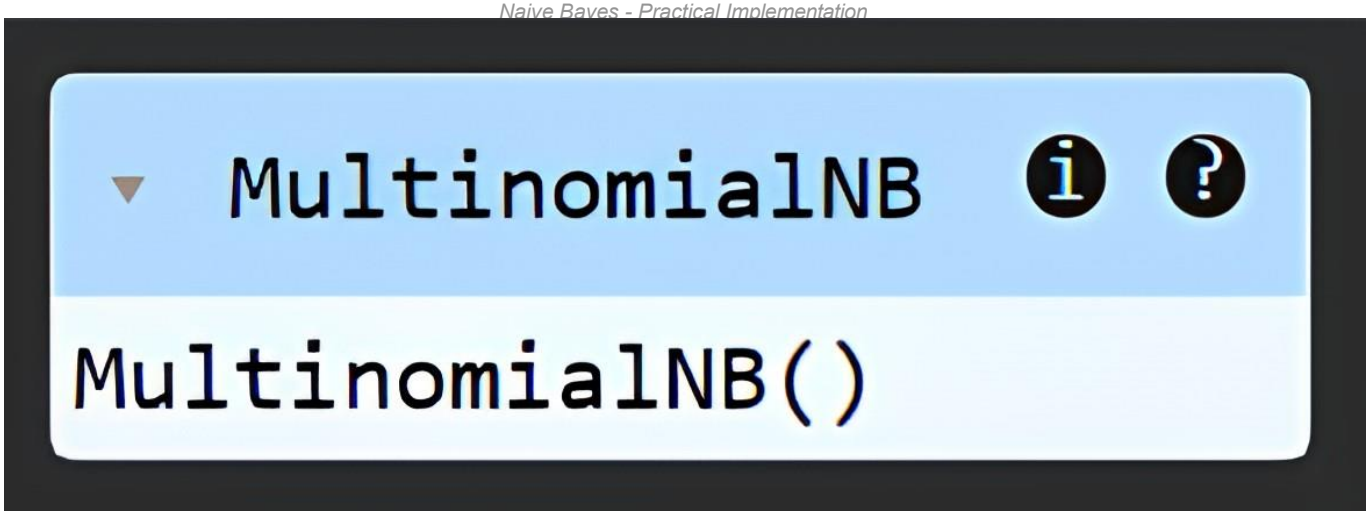
```
Text data vectorized using TF-IDF.
```

 # Trains a Multinomial Naive Bayes classifier on the TF-IDF features.

```
model=MultinomialNB()
model.fit(X_train_vec,y_train)
print("Model training completed.")
```

Output:

```
Model training completed.
```



Step 6: Model Evaluation

 # Predicts on test data and evaluates model

```
y_pred = model.predict(X_test_vec)
y_prob = model.predict_proba(X_test_vec)[:, 1]
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nClassification Report:\\n", classification_report(y_test, y_pred))
```

Output:

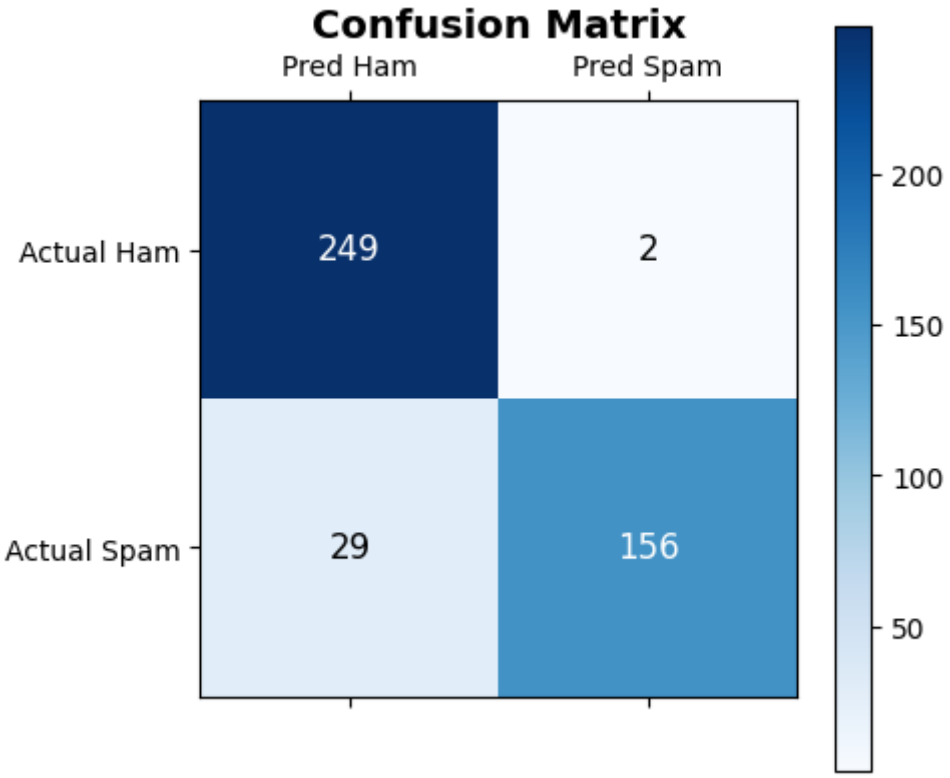
```
Accuracy: 0.9288990825688074
Classification Report:
precision    recall  f1-score   support
0(ham)       0.90      0.99      0.94       251
1(spam)       0.99      0.84      0.91       185
accuracy                          0.93       436
macro avg       0.94      0.92      0.93       436
weighted avg       0.93      0.93      0.93       436
```

 # Plots the confusion matrix with values to visualize correct and incorrect predictions

```
cm = confusion_matrix(y_test, y_pred); plt.matshow(cm, cmap='Blues')
for (i, j), val in np.ndenumerate(cm):
    plt.text(j, i, val, ha='center', va='center', color='white' if val > cm.max()/2 else
    'black', fontsize=12)
plt.xticks([0,1], ['Pred Ham','Pred Spam'])
plt.yticks([0,1], ['Actual Ham','Actual Spam']); plt.colorbar();
plt.show(); print("Confusion matrix displayed.")
```

Output:

```
Confusion matrix displayed.
```

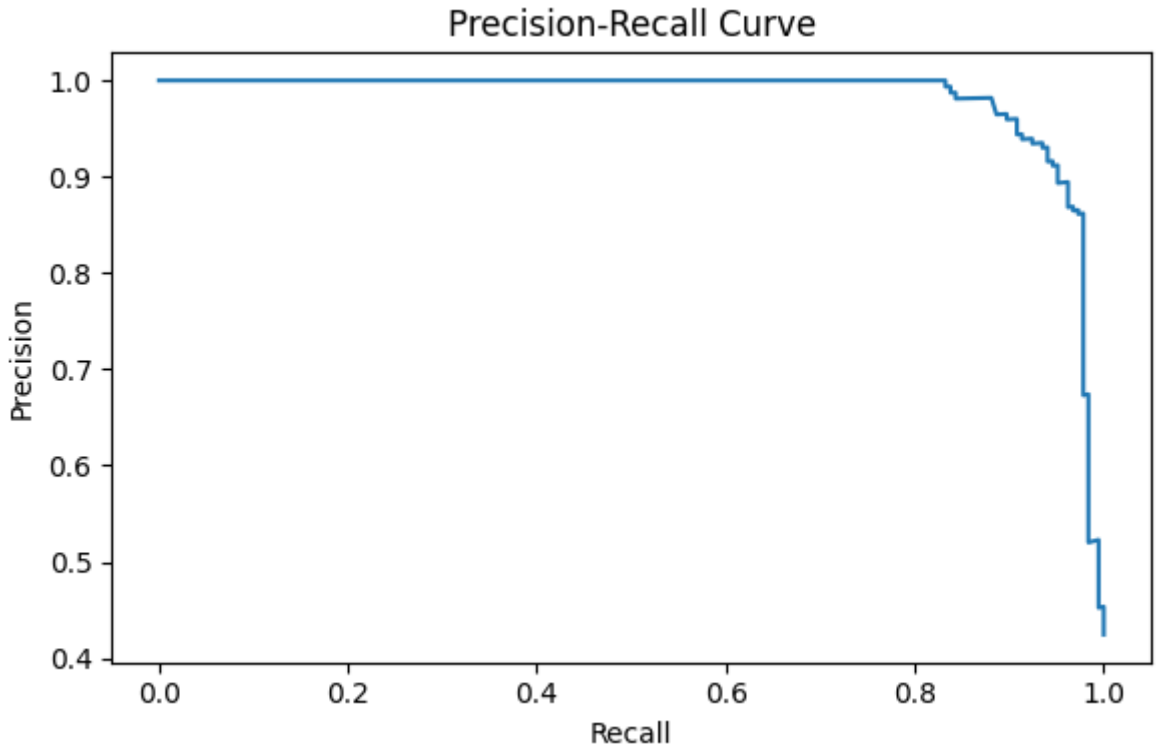


 # Plots the Precision-Recall curve for the model

```
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.figure(figsize=(6,4)); plt.plot(recall, precision); plt.title('Precision-Recall
    Curve'); plt.xlabel('Recall');plt.ylabel('Precision'); plt.tight_layout(); plt.show()
print("Precision-Recall curve displayed.")
```

Output:

```
Precision-Recall curve displayed.
```



 # Highlights words based on their contribution to spam/ham prediction using color intensity

```
def color_text(words, contributions):
    max_val = max(abs(c) for c in contributions) or 1; html = ""
    for w, c in zip(words, contributions):
        intensity = int(255 * (abs(c) / max_val))
        if c > 0: color = f"rgb(255,{255-intensity},{255-intensity})"
        else: color = f"rgb({255-intensity},255,{255-intensity})"
        html += f"{w} "
    return html
print("Text contribution visualization function defined.")
```

Output:

```
Text contribution visualization function defined.
```

 # Computes and displays precision and recall of the model on test data

```
def show_precision_recall(model, X_test_vec, y_test):
    preds = model.predict(X_test_vec)
    precision = precision_score(y_test, preds) ;recall = recall_score(y_test, preds)
print("Precision and recall calculation function ready.")
```

Output:

```
Precision and recall calculation function ready.
```

 # Performs precision-recall evaluation and analyzes 'n' random test messages

```
def analyze_multiple(model, vectorizer, X_test, y_test, n=5):
    show_precision_recall(model, X_test_vec, y_test)
    for i in range(n):
        analyze_random_message(model, vectorizer, X_test, y_test)
print("Multiple message analysis function defined.")
```

Output:

```
Multiple message analysis function defined.
```

 # Runs evaluation and displays analysis of 5 random test messages

```
analyze_multiple(model, vectorizer, X_test, y_test, n=5)
print(" MODEL PERFORMANCE ON TEST DATA")
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

Output:

```
MODEL PERFORMANCE ON TEST DATA
Precision: 0.9873
Recall    : 0.8432
Choose one random test sample.
```

Output:

Category	Message
ham	It's getting me down just waiting around.
spam	Text and meet someone today. Reply with your name and age.
ham	I.ll give her once i have it. Plus she said grinule greet you whenever we speak
spam	Congratulations U can claim 2 VIP row A Tickets 2 C Blu in concert in November or Blu gi..
ham	thinking about you.

Output:

```
Message Analysis
Actual:
Predicted:
Spam Probability:
```

Interactive Features (Static View)

Interactive Feature: Evaluation Sample Analysis

Clicking an evaluation sample shows the model prediction and word influence analysis. Below are all 5 test samples with results:

#	Category	Message	Predicted	Spam Prob
1	ham	It's getting me down just waiting around.	ham	0.0408
2	spam	Text and meet someone today. Reply with your name and age.	spam	0.5772
3	ham	I'll give her once i have it. Plus she said greet you when..	ham	0.0322
4	spam	Congratulations U can claim 2 VIP Tickets to C Blu in conc..	spam	0.9309
5	ham	thinking about you.	ham	0.0670

Interactive Feature: Random Message Prediction

The simulation tests the model on random messages:

#	Message	Actual	Predicted
1	Working on the project now.	Ham	Ham
2	WINNER! You won a FREE gift!	Spam	Spam
3	Lunch at 1pm?	Ham	Ham